INFORME DE ACTIVIDAD AUTONOMA DE LABORATORIO - SESIÓN 8

Mensajería Asíncrona entre Servicios (Apache Kafka, Eventos y Desacople)

1. DATOS DEL ESTUDIANTE

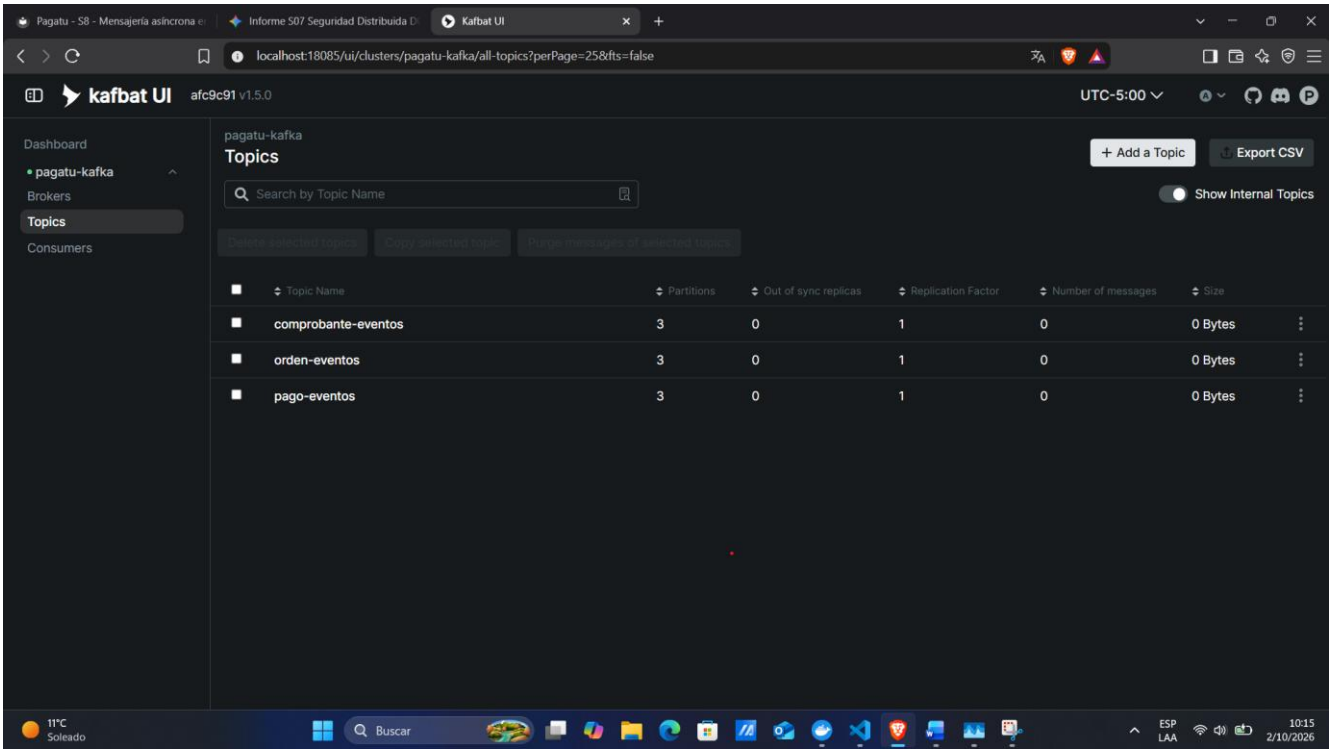
Nombre y Apellidos:	Witmer Jhetly Mamani Cahuana
Equipo de Trabajo:	Equipo 01
Sesión:	S08 Mensajería asíncrona entre servicios
Rol o Aporte Realizado:	Despliegue del broker Apache Kafka en modo KRaft y Kafka UI con Docker en DEV; creación explícita de los topics orden-eventos y pago-eventos con 3 particiones; construcción integral de pagatu-pago-ms con persistencia en PostgreSQL, consumo de orden.creada y emisión de pago.validado tras commit; adecuación de pagatu-orden-ms para publicar eventos y actualizar estado a PAGADA; validación de desacople apagando el consumidor y comprobando acumulación de lag; e implementación del evento autónomo comprobante.solicitado hacia pagatu-cliente-ms.
Enlace del Repositorio en GitHub:	https://github.com/Witmer17/Mensajeria_Asincrona_Entre_Servicios.git

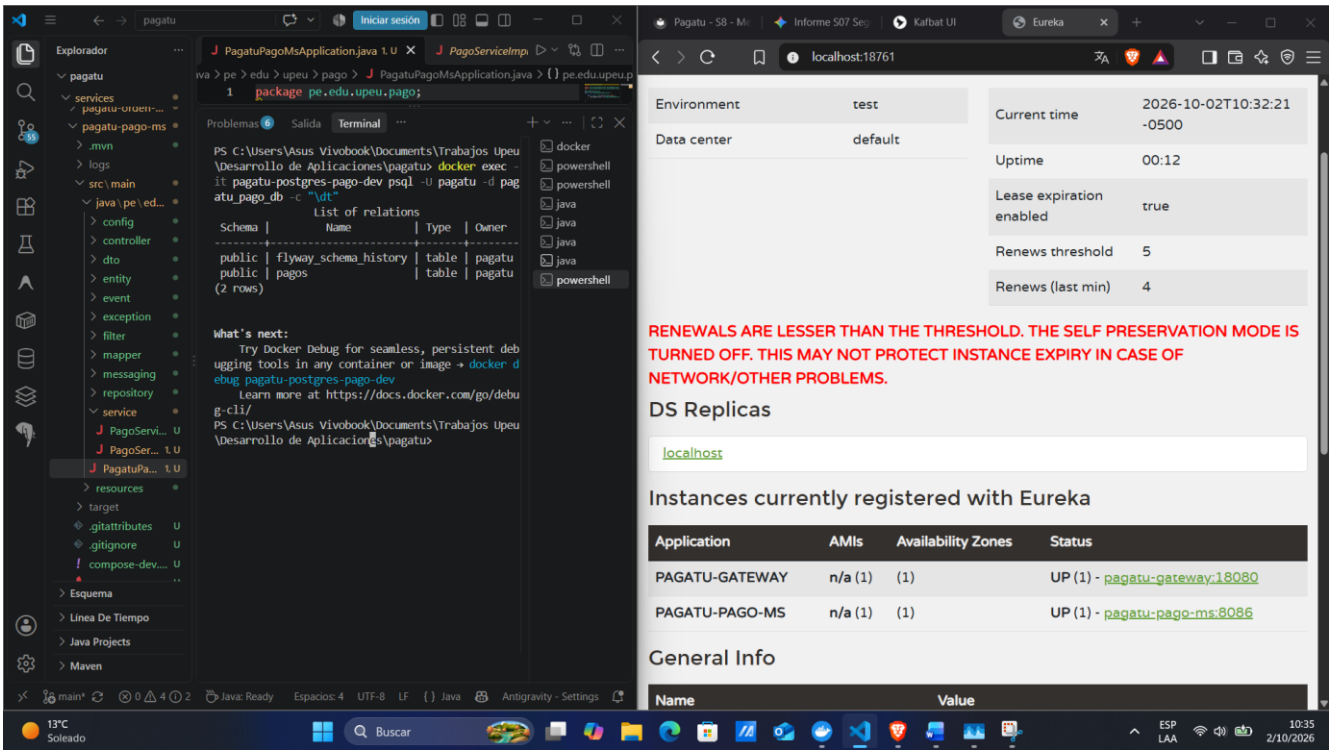
2. EVIDENCIA TÉCNICA Y RESULTADOS

Bloque 1: Kafka y pagatu-pago-ms Construidos (Infraestructura y Persistencia)

Levanté el broker Apache Kafka (apache/kafka:4.3.1) en modo KRaft (puertos 19092 para clientes Spring en el host y 9092 para redes internas de Docker) junto con la interfaz gráfica Kafka UI en el puerto 18085. Se configuró de forma intencional KAFKA_AUTO_CREATE_TOPICS_ENABLE: false para exigir la definición manual de topics, creándose orden-eventos y pago-eventos con 3 particiones y factor de replicación 1.

Asimismo, desarrollé el cuarto microservicio del ecosistema, pagatu-pago-ms, levantándolo en el puerto DEV 8086 y respaldado por su contenedor PostgreSQL (pagatu-postgres-pago-dev, puerto host 15435, base de datos pagatu_pago_db). El servicio se conecta a pagatu-config para traer su configuración externa, se registra exitosamente en Eureka y ejecuta mediante Flyway la migración V1__create_pagos.sql generando la tabla pagos con la restricción de unicidad uk_pagos_orden sobre orden_id.

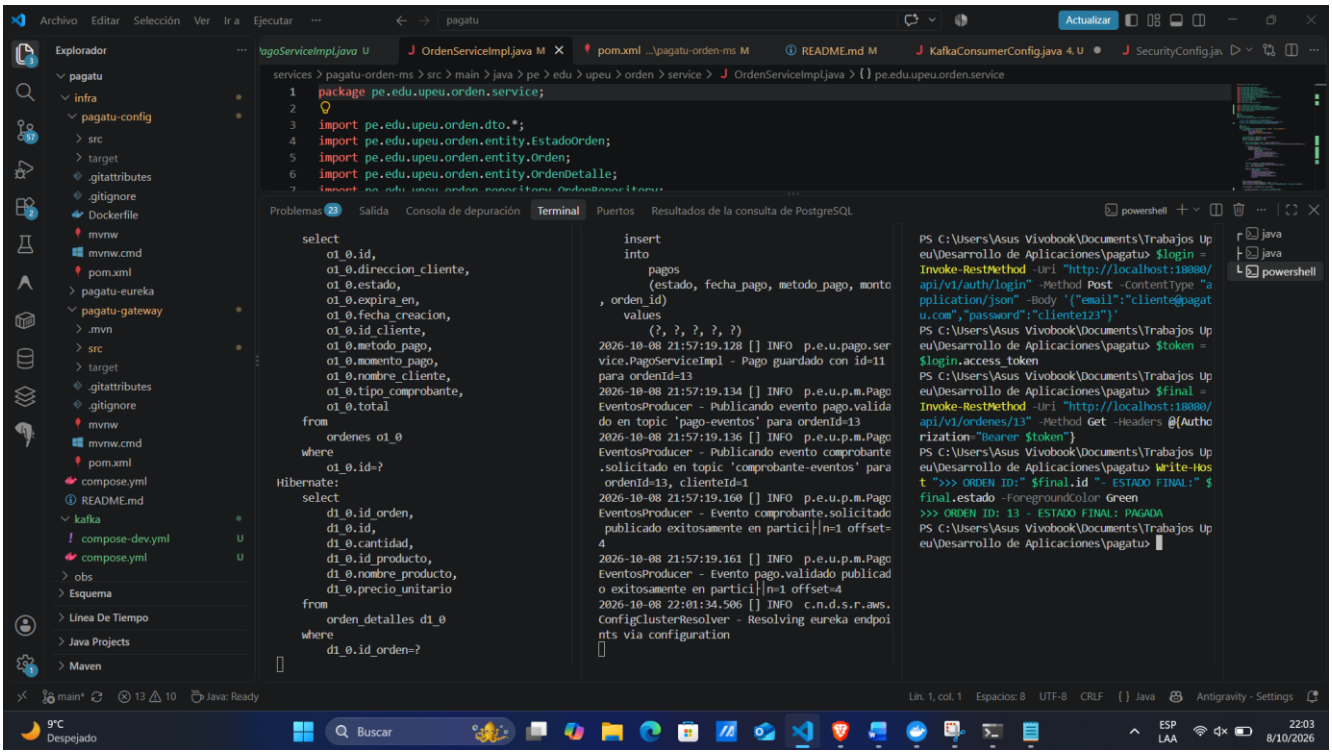


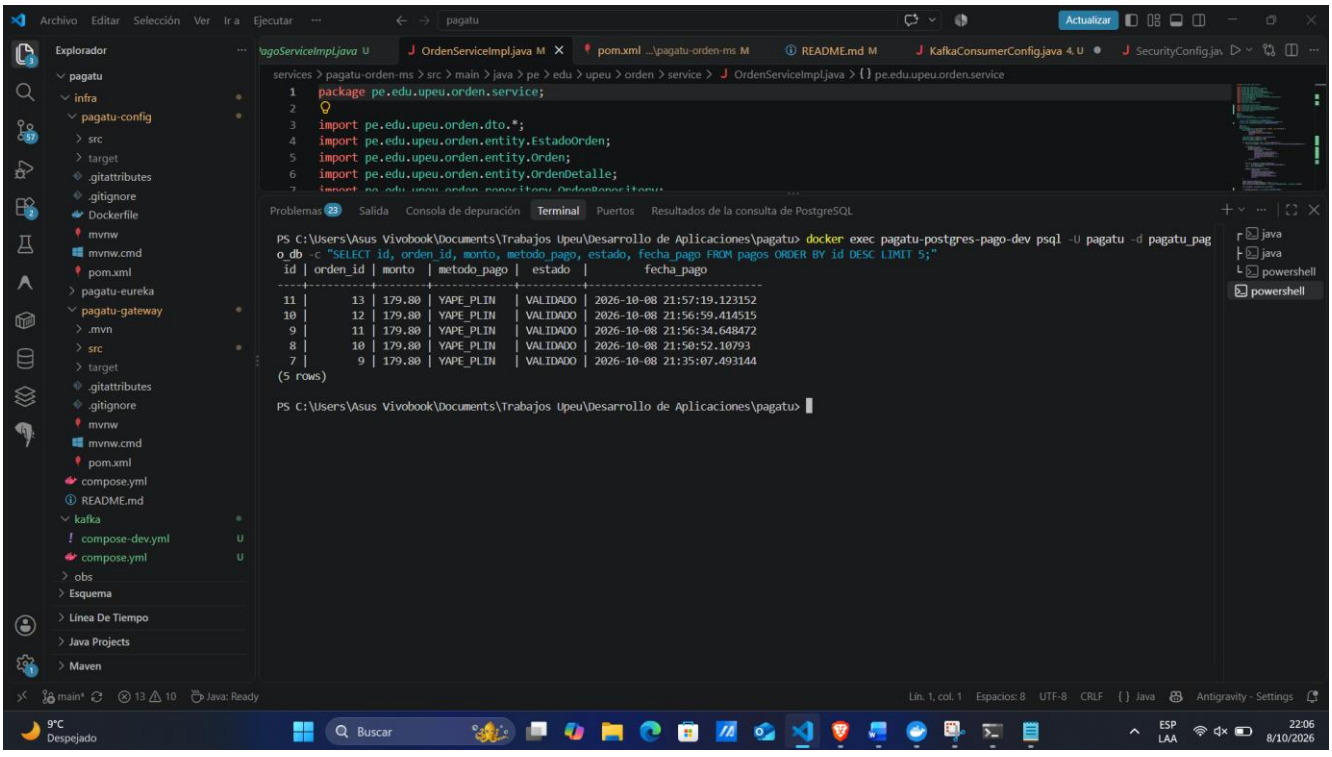


Bloque 2: Eventos entre pagatu-orden-ms y pagatu-pago-ms (Flujo Completo)

Se implementó el ciclo de vida asíncrono completo. Al recibir un request POST /api/v1/ordenes con token de CLIENTE, pagatu-orden-ms persiste la orden en estado PENDIENTE_PAGO y, mediante TransactionSynchronizationManager.registerSynchronization (afterCommit), emite el evento orden.creada al topic orden-eventos usando ordenId como message key.

El microservicio pagatu-pago-ms, mediante su @KafkaListener en OrdenEventosConsumer, consume el mensaje, deserializa el payload con ErrorHandlingDeserializer y procesa el cobro simulado creando el registro de pago en estado VALIDADO. Inmediatamente después de confirmar la transacción, emite pago.validado al topic pago-eventos. Finalmente, pagatu-orden-ms consume dicha confirmación y actualiza de manera reactiva el estado de la orden a PAGADA sin que exista ningún acoplamiento síncrono ni llamada HTTP directa entre ambos microservicios.

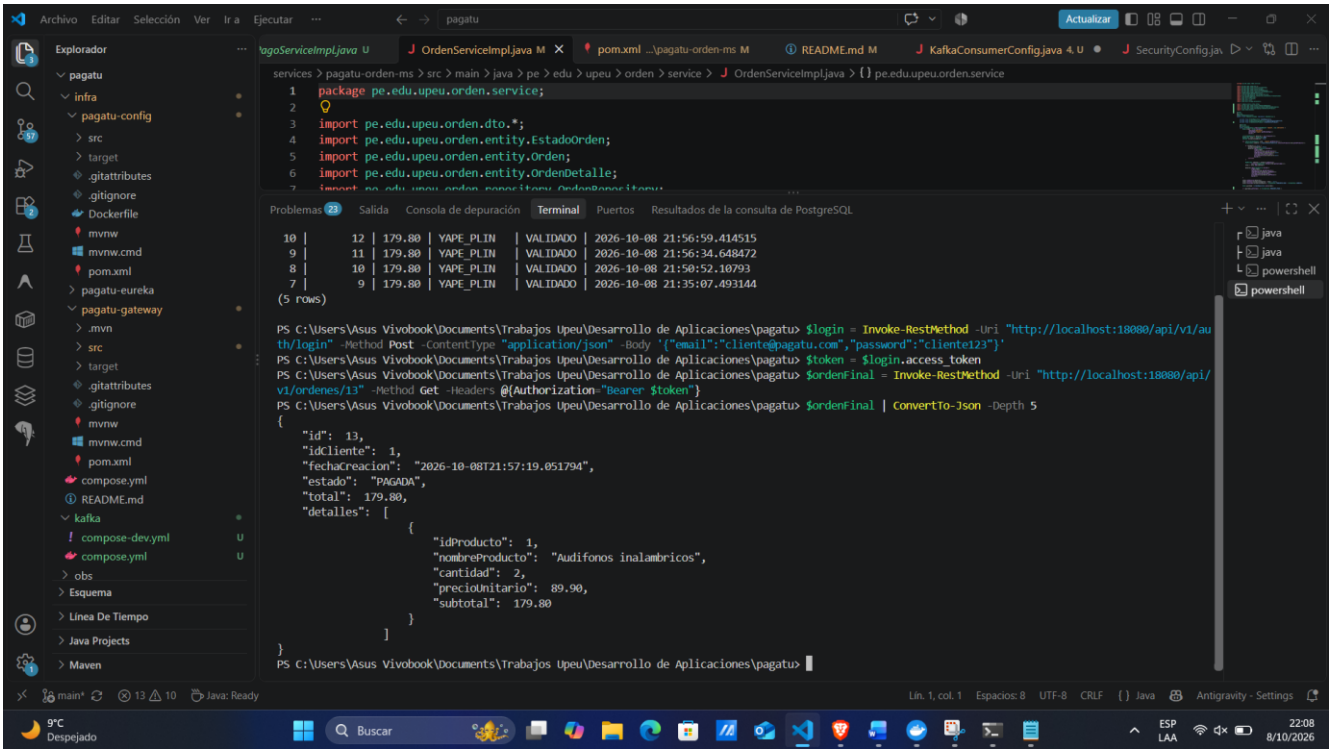


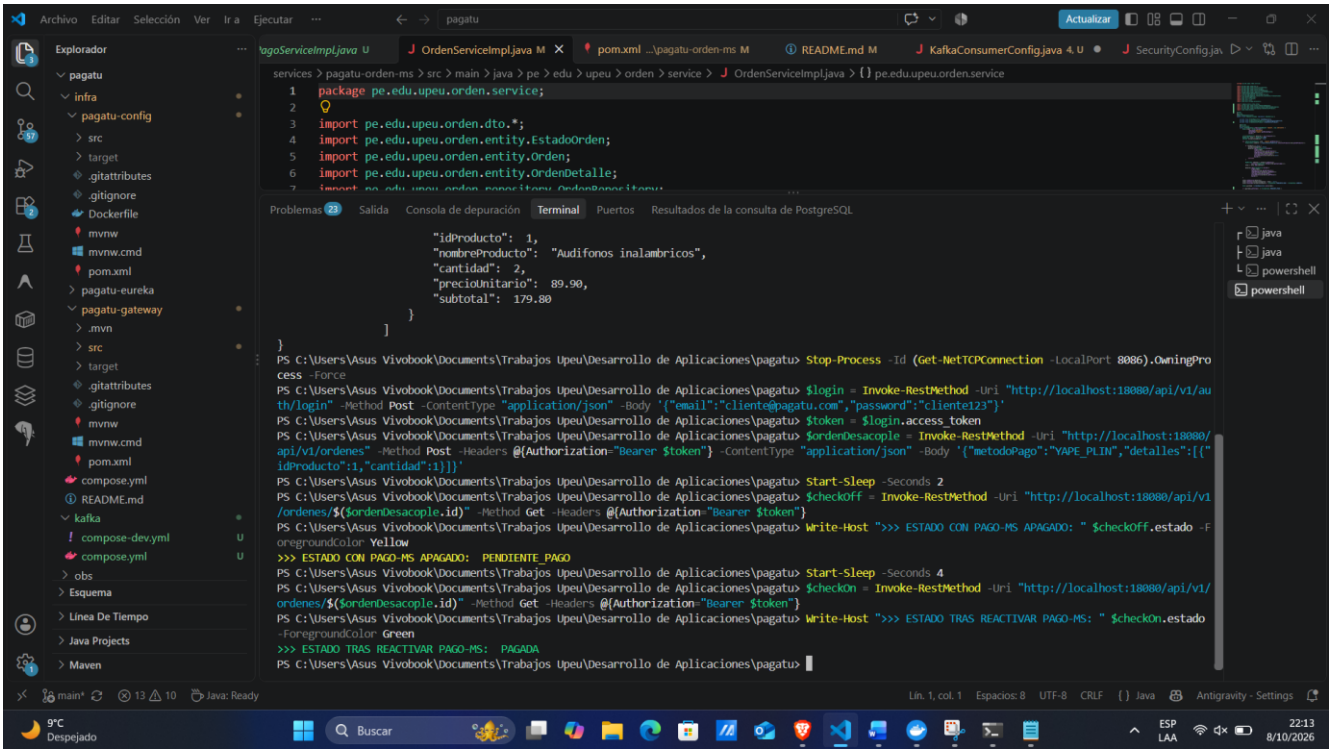


Bloque 3: Desacople Temporal y Resiliencia Evidenciada

Para evidenciar el desacople temporal real, detuve el microservicio pagatu-pago-ms (apagado en consola). En este escenario, procedí a registrar una nueva orden mediante el Gateway. pagatu-orden-ms procesó la solicitud y retornó 201 Created de inmediato con la orden en estado PENDIENTE_PAGO, demostrando que la caída del cobrador no bloquea al servicio emisor.

En Kafka UI, en la sección de Consumer Groups, se comprobó que el grupo pagatu-pago-ms acumuló un lag de 1 mensaje pendiente en el topic orden-eventos. Al volver a encender pagatu-pago-ms, el servicio reanudó la lectura desde su último offset confirmado, consumió el evento pendiente, registró el pago en la base de datos y publicó pago.validado, llevando la orden automáticamente a PAGADA y restableciendo el lag del grupo a 0 sin pérdida de mensajes.

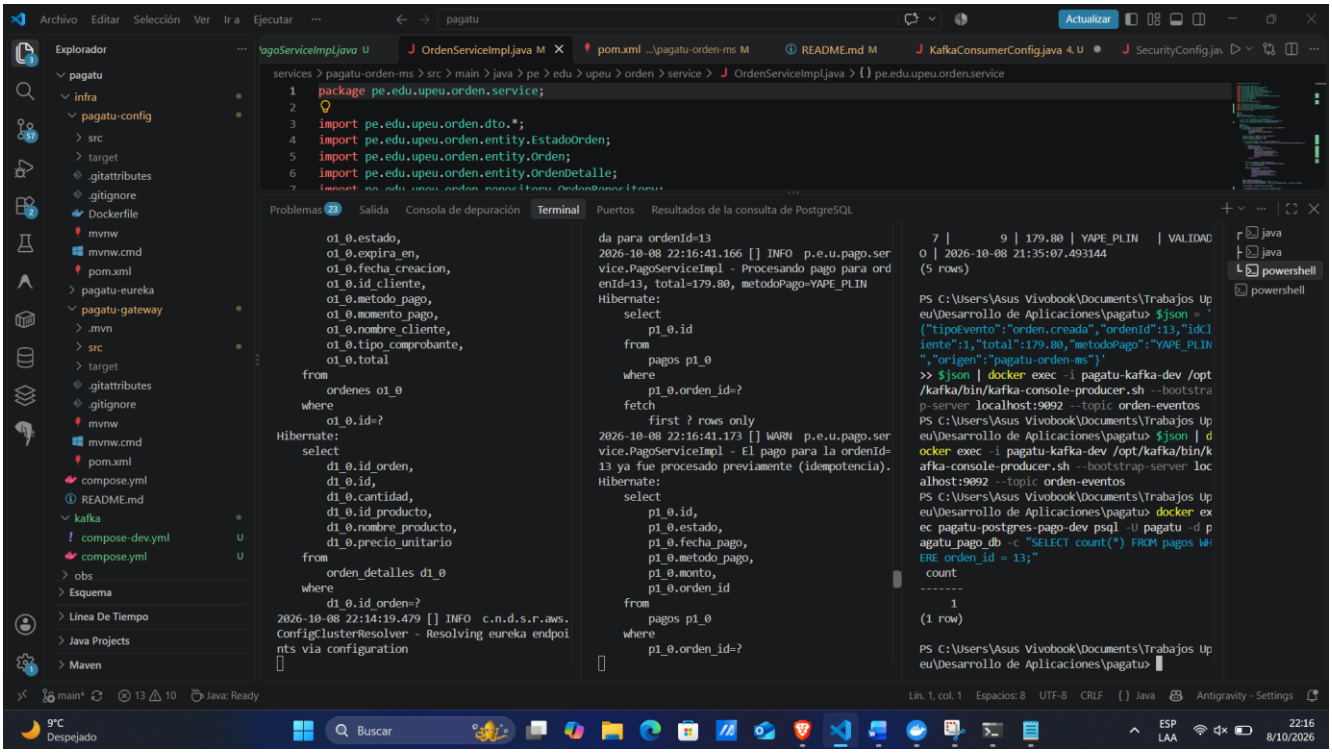




Bloque 4: Evento de Negocio Adicional Autónomo (comprobante.solicitado)

Como actividad autónoma individual, integré un tercer evento de negocio denominado comprobante.solicitado publicado por pagatu-pago-ms inmediatamente después de confirmar un pago exitoso, y consumido de forma asíncrona por pagatu-cliente-ms para simular la emisión del comprobante electrónico y su notificación por WhatsApp al cliente.

Se configuró el topic comprobante-eventos con 3 particiones y se probó el desacople apagando pagatu-cliente-ms mientras se realizaban pagos: los eventos se acumularon en el broker con lag ascendente y se procesaron de forma íntegra apenas el microservicio de clientes reanudó su operación, comprobando la escalabilidad del patrón pub/sub en el clúster.



Documentación Oficial de Contratos de Eventos

Topic: orden-eventos | Contrato del Evento: orden.creada

Productor: pagatu-orden-ms | Consumidor: pagatu-pago-ms | Key: ordenId | Publicación: Tras confirmar registro de orden en PENDIENTE_PAGO.

Campo	Tipo	Descripción
tipoEvento	string	Constante 'orden.creada' que identifica la acción producida.
ordenId	number	Identificador único de la orden; actúa como message key para ordenamiento.
idCliente	number	ID del cliente autenticado extraído de los claims del JWT.
total	number	Monto total calculado tras validar precios en catálogo.
metodoPago	string	Modalidad seleccionada (ej. TARJETA, YAPE_PLIN, PAGO_EFECTIVO).
origen	string	Servicio emisor ('pagatu-orden-ms').
timestamp	number	Marca de tiempo epoch en milisegundos de la emisión.

Topic: pago-eventos | Contrato del Evento: pago.validado

Productor: pagatu-pago-ms | Consumidor: pagatu-orden-ms | Key: ordenId | Publicación: Tras guardar pago en estado VALIDADO.

Campo	Tipo	Descripción

tipoEvento	string	Constante 'pago.validado' que confirma la aprobación del cobro.
ordenId	number	Identificador de la orden a la cual corresponde el pago liquidado.
monto	number	Monto validado cobrado al usuario.
estado	string	Estado del pago procesado ('VALIDADO').
origen	string	Servicio emisor ('pagatu-pago-ms').
timestamp	number	Marca de tiempo epoch en milisegundos de la confirmación.

3. ERROR O HALLAZGO TÉCNICO

¿Qué pasó en la práctica?: Al arrancar pagatu-pago-ms y enviar la primera orden, el microservicio se conectaba a Kafka pero jamás consumía ningún evento de orden-eventos, manteniéndose el lag en 1 indefinidamente.

¿Por qué fallaba?: Revisando minuciosamente los logs y la configuración de infra/pagatu-config/config-repo/pagatu-pago-ms-dev.yml, descubrí que en spring.kafka.consumer.properties.spring.json.trusted.packages había colocado pe.edu.upeu.orden.event en lugar de pe.edu.upeu.pago.event. Debido a que el productor desactiva las cabeceras de tipo (spring.json.add.type.headers: false), Jackson intentaba deserializar el payload en la clase por defecto especificada, pero la validación de paquetes seguros de Spring Kafka bloqueaba la instanciación por desconfianza en el paquete configurado.

¿Cómo lo solucioné?: Corregí la propiedad trusted.packages hacia pe.edu.upeu.pago.event y configuré el ErrorHandlingDeserializer con el delegado JacksonJsonDeserializer adecuadamente. Al reiniciar el servicio, el consumidor tomó inmediatamente el evento pendiente desde el offset acumulado y procesó el pago con éxito.

4. REFLEXIÓN TÉCNICA

(Respuesta en 5 a 8 líneas al requerimiento de la sección 4.3.1 de la guía)

Para que pagatu-pago-ms pudiera cobrar sin llamar de vuelta a pagatu-orden-ms, el evento orden.creada tuvo que llevar los datos suficientes del estado del hecho: ordenId, idCliente, total y metodoPago. Si el evento hubiera contenido únicamente el ordenId como simple aviso, pagatu-pago-ms habría tenido que hacer una llamada síncrona por HTTP (con Feign) a pagatu-orden-ms para averiguar cuánto cobrar y con qué método. Al hacer eso, se destruiría por completo la independencia temporal que buscamos con la mensajería asíncrona: si pagatu-orden-ms estuviera ocupado o caído en ese instante, el cobro fallaría y los servicios volverían a quedar rígidamente acoplados. Diseñar eventos ricos en datos permite que cada consumidor trabaje de forma 100% autosuficiente y autónoma.

5. ANEXO: FEEDBACK DE LA SESIÓN

¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?

Comprender el desacoplamiento temporal que proporciona Apache Kafka: un servicio productor puede seguir respondiendo al usuario y publicar un hecho sin importarle si el consumidor está activo o apagado en ese momento, garantizando que nada se pierde gracias a los offsets y particiones.

¿Qué punto de la clase te resultó más confuso o te dejó con dudas?

La diferencia entre las direcciones INTERNAL (kafka:9092 para contenedores Docker) y EXTERNAL (localhost:19092 para aplicaciones que corren directamente en la máquina anfitriona) en KAFKA_ADVERTISED_LISTENERS.

¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?

¿Cómo se implementa en código la idempotencia cuando Kafka entrega un mensaje duplicado (at-least-once delivery), y cómo se coordina la compensación si un pago falla en el patrón Saga?

Sobre tu nivel de comprensión de la clase de hoy:

[X] ¡Entendido! - Lo domino y podría explicarlo.

[ ] Más o menos. - Entendí la idea general, pero tengo dudas.

[ ] Necesito ayuda. - Me siento perdido/a con este tema.

¿Cómo puedo ayudarte a comprender mejor el tema?

Con ejemplos paso a paso sobre cómo estructurar la idempotencia en la base de datos para evitar cobros dobles cuando se reintentan mensajes en Kafka.

Pensando en tu participación y esfuerzo en la clase de hoy, ¿cómo te autoevaluarías?

[X] Muy Comprometido/a: Me esforcé al máximo.

[ ] Comprometido/a: Sé que podría haberme esforzado un poco más.

[ ] Poco Comprometido/a: Hoy no di mi mejor esfuerzo.

Mi satisfacción con la clase fue (del 1 al 10):

Mi satisfacción fue de 8.